40 Days of LeetCode Challenge

40 days, 40 problems
DAY :7

LEETCODE PROBLEM NO. 231 : Power of Two

Problem Link : https://leetcode.com/problems/power-of-two/description/
Easy · Bit Manipulation ·
STEP BY STEP SOLUTION(The following problem can be solved using 2 efficient & optimized methods) :-
note :- Apart from this solution there are many possible approaches as well

Step 1. Understand the question :-

We are given an integer n. We need to return true if n is a power of two and false otherwise.

A number is a power of two if it can be written as: [n = 2^x] where (x) is a non-negative integer.

Some powers of two are:

Power	Value
(2^0)	1
(2^1)	2
(2^2)	4
(2^3)	8
(2^4)	16
(2^5)	32

NOTE:-

1 is a power of two because ((2^0)= 1). Zero and negative numbers are not powers of two.

Examples:

Input	Output	Reason
1	true	((2^0)= 1)
2	true	((2^1)= 2)
16	true	((2^4)= 16)
3	false	Not a power of two
0	false	Powers of two are positive
-8	false	Negative numbers aren't powers of two

Step 2. Translating it into simpler language with examples :-

Example 1:
Input: n = 1
Output: true
Explanation: 2^0 = 1

Example 2:

Input: n = 16
Output: true
Explanation: 24^ = 16

Example 3:
Input: n = 3
Output: false

Example 4: We need to check whether a number belongs to this sequence:

1, 2, 4, 8, 16, 32, 64, 128, ...

Instead of repeatedly dividing the number by two, our chosen approach counts the number of 1 bits in its binary representation.

A positive power of two always has exactly one 1 in binary.

- If n = 1, return true.
- If n = 16, return true.
- If n = 3, return false, because 3 isn't in the sequence.
- If n = 0, return false.
- If n = -8, return false.

Number: 8 — Power of two

Binary: 1000

Number of set bits = 1

Number: 7 — Not a power of two

Binary: 0111

Number of set bits = 3

Step 3. Identify the pattern

The key concept is binary representation and bit counting. In binary, every power of two has exactly one set bit:

Decimal	Binary	Set bits
1	0001	1
2	0010	1
4	0100	1
8	1000	1
10	1010	2
12	1100	2

Java provides the built-in method Integer.bitCount(n), which returns the number of set bits (1s) in an integer's binary representation.

Therefore, if n is positive and its bit count equals 1, it is a power of two.

OR

There are two useful patterns for this problem.

Pattern A: Repeated division

A power of two can be divided by 2 repeatedly until we reach 1, without ever getting a remainder.

For example:

[16 ÷ 2 = 8 → 4 → 2 → 1]

But for 3, dividing by 2 gives a remainder, so it isn't a power of two.

Pattern B: Binary representation

Computers represent numbers using binary, which contains only 0 and 1.

Look at these powers of two:

Decimal	Binary
1	0001
2	0010
4	0100
8	1000
16	10000

NOTE:-

Every positive power of two has exactly one 1 in its binary representation.

For comparison:

- 8 is 1000 — one set bit.
- 10 is 1010 — two set bits, so it's not a power of two.
- 16 is 10000 — one set bit.

This observation leads to a very efficient bit-manipulation solution.

Step 4. Build the logic

Here's the approach we'll follow.

Algorithm

1. Check whether n is greater than zero.
2. Count the set bits using Integer.bitCount(n).
3. If the count equals 1, return true.
4. Otherwise, return false.

OR

1. Check that n is positive.
2. Check whether (n & (n - 1)) == 0.
3. Return true only if both conditions hold.

We can solve the problem without a loop by using the bitwise AND operator &.

For any positive power of two, the expression n & (n - 1) equals zero.

NOTE:-

Subtracting 1 from a power of two flips its only 1 bit to 0 and turns the trailing zeros into ones.

For example:

```
n       = 8  = 1000
n - 1   = 7 = 0111
            --------
AND       = 0000
```

So 8 & 7 equals 0.

NOTE:-

0 & (-1) is also zero. That's why we must first ensure that n > 0.

Pseudocode

```
START
  IF n <= 0
    RETURN false

  COUNT = number of set bits in n

  IF COUNT == 1
    RETURN true
  ELSE
    RETURN false
END
```

OR

```
ALGORITHM IsPowerOfTwo(n)
BEGIN
  IF n > 0 AND (n AND (n - 1)) = 0 THEN
    RETURN true
  ELSE
    RETURN false
  END IF
END
```

Step 5. Code in Java

This is the standard arithmetic solution for LeetCode 231.

```
class Solution {
   public boolean isPowerOfTwo(int n) {
      return n > 0 && Integer.bitCount(n) == 1;
   }
}
```

OR

```
class Solution {
   public boolean isPowerOfTwo(int n) {
      return n > 0 && (n & (n - 1)) == 0;
   }
}
```

How to submit

1. Open LeetCode Problem 231. Power of Two
2. Select Java as the programming language.
3. Enter the method in the provided Solution class, replacing the existing method if necessary.
4. Click Run to test the examples.
5. Click Submit after the tests pass.

Step 6. Explanation & dry-run

Explanation of the code :-

Line 1

class Solution {

Defines the class expected by LeetCode.

Line 2

public boolean isPowerOfTwo(int n) {

Defines a method that receives an integer n and returns a Boolean value: true or false.

Line 3

return n > 0 && Integer.bitCount(n) == 1;

This line combines two conditions using the logical AND operator &&.

- n > 0 checks that the number is positive.
- Integer.bitCount(n) counts the 1 bits in its binary representation.
- == 1 checks whether exactly one bit is set.

- && requires both conditions to be true.

NOTE:-

Java's short-circuit evaluation means that if n > 0 is false, the second condition doesn't need to be evaluated.

Dry run

Let's test n = 16.

Step	Operation	Result
1	n > 0	true
2	Binary of 16	10000
3	Integer.bitCount(16)	1
4	1 == 1	true
5	Final return	true

Now let's test n = 12.

Step	Operation	Result
1	n > 0	true
2	Binary of 12	01100
3	Integer.bitCount(12)	2
4	2 == 1	false
5	Final return	false

And if n = 0, the first condition is false, so the method immediately returns false.

OR

return n > 0 && (n & (n - 1)) == 0;

Part A: n > 0 :- A power of two must be positive. This rejects 0 and all negative numbers.

Part B: n - 1 :- Subtracting 1 from a power of two changes its only set bit and all trailing zeros.

For example:

8 = 1000
7 = 0111

Part C: n & (n - 1):- The bitwise AND operator compares each binary position. The result is 1 only where both operands contain 1.

```
 1000
& 0111
------
 0000
```

Part D: == 0 :- This checks whether the bitwise operation produced zero.

Part E: && :- Both conditions must be true: the number must be positive, and the bitwise result must equal zero.

Dry run A: n = 16

```
n      = 16    = 10000
n - 1  = 15    = 01111
                --------
AND result     = 00000
```

- n > 0 → true
- (n & (n - 1)) == 0 → true
- true && true → true
- Output: true

Dry run B: n = 3

```
n      = 3     = 0011
n - 1  = 2     = 0010
                -------
AND result     = 0010
```

- n > 0 → true
- (n & (n - 1)) == 0 → false, because the result is 2, not 0.
- true && false → false
- Output: false

Dry run C: n = 1

```
n      = 1     = 0001
n - 1  = 0     = 0000
                -------
AND result     = 0000
```

- n > 0 → true
- (n & (n - 1)) == 0 → true
- Final result → true
- This confirms that (2^0 = 1) is handled correctly.

Dry run D: n = 0

- n > 0 → false
- Java's && short-circuits, so the right-hand condition isn't evaluated.
- Final result → false

Step 6. Analyze complexity and optimization

- Time complexity: (O(1))
- Auxiliary space complexity: (O(1))

NOTE:-

Java's int has a fixed width of 32 bits. Integer.bitCount() processes that fixed-size representation, so its work is constant with respect to the input value.

Can we optimize it further?

For the Java int type, this is already an excellent constant-time solution. The bitwise approach (n & (n - 1)) == 0 is another constant-time option, but neither should be assumed to run faster on every LeetCode submission. Actual measured milliseconds can fluctuate.

SCREENSHOT:-



OR

